基于改良模拟退火算法和统计方法的生产过程决策

摘　要

本文通过对生产过程的各个阶段的研究，制定出合理的产品检测、装配和拆解策略，可以帮助企业最大限度地降低成本获得更高的利润，同时重复使用零配件以减少废物的产生，也增强消费者的购物体验。

针对问题一，首先采用二项分布来设计合理的抽样检测方案，当样本量足够大时，根据独立同分布的中心极限定理，用正态分布去近似二项分布，最后利用统计检验求出最小的样本容量。在 95%的信度下认定零配件次品率超过 10%拒收这批零配件时的样本容量为 **98**，在 90%的信度下认定零配件次品率不超过 10%接收这批零配件的样本容量为 **59**。

针对问题二，首先对数据进行量化，以 1 和 0 来表示零配件 1、零配件 2 和成品是否检测以及不合格成品是否拆解。题目中共列举 6 种情况，针对每一种情况，这 4 个自变量都可以构成 16 种方案，本文根据检测情况的不同而得出实际的成品次品率，假设购买的零配件初始值均为 100，然后根据题目中所给参数，依次算出每种方案的购买成本、检测成本、装配成本、拆解成本、调换成本、销售收入，最后以最大利润为目标，比较两种计算拆解次数的方法，求出每种方案的利润，并据此画出拆解次数与利润的关系图排序比较即可得最大利润及其对应的方案。其中，情况 1 的决策方案为：**零配件 1 不检测-零配件 2 不检测-成品不检测-不合格品拆解**，且每一个零配件经历的拆解次数最多为 4 次，此时最大利润为 **1957**，单价为 **19.57 元/件**，其余结果见表 3。

针对问题三，在问题二的基础上增加了零配件和半成品，将问题二建立的模型进行扩充，建立决策矩阵，使用模拟退火算法，通过多次迭代以及降温处理最后得到最大利润值和决策方案。决策方案为：**零配件 1 到 8 不检测-半成品 1 不检测-半成品 2 检测-半成品 3 不检测-半成品 1 拆解-半成品 2 不拆解-半成品 3 拆解-成品检测-不合格品拆解**，且每一个零配件经历的拆解次数最多为 5 次，此时最大利润为 **6352**，单价为 **63.52 元/件**。

针对问题四，将抽样检测方法得到的次品率结合第一问的分析，给出次品率的置信区间，分别将该置信区间的上限看作最坏情况，将置信区间下限看作最优情况，然后利用问题二和问题三的模型及求解方法给出两种情况下的决策方案，见表 7 和表 8。

关键词：z 检验　枚举法　改良模拟退火算法　置信区间

一、 问题重述

1.1 问题背景

某企业生产的某种电子产品非常畅销，该产品的生产是购买零配件 1、零配件 2 装配成成品。在装配过程中，若某一个零配件不合格，则装配成的成品一定不合格；若两个零配件都合格，则装配成的成品不一定合格。企业可以选择将不合格的成品报废或拆解，如果选择拆解，拆解过程中不会对零配件造成损坏，但需要支出额外的拆解费用。因此，通过合理安排检测、装配和拆解策略，企业可以最大限度地降低成本获得更高的利润，也可以提高消费者买到的产品的质量，同时拆解不合格成品并重复使用零配件的做法有助于减少废物产生，符合循环经济的理念。一举多得，该问题具有重要的研究意义。

1.2 问题重述

根据题目所给信息建立数学模型，回答下列问题：

问题一：若某批零配件的次品率不会超过标称值 10%，根据以下两种具体情况，为企业设计检测次数尽可能少的抽样检测方案。

（1）在 95%的信度下认定零配件次品率超过 10%，则拒收这批零配件。

（2）在 90%的信度下认定零配件次品率不超过 10%，则接收这批零配件。

问题二：针对表 1 的 6 种具体情况为企业生产的各个阶段做出决策，并说明决策的依据和对应的指标结果。

表 1 企业在生产中遇到的情况（问题二）

情况	零配件 1			零配件 2			成品			不合格成品		
	次品率	购买单价	检测成本	次品率	购买单价	检测成本	次品率	装配成本	检测成本	市场售价	调换损失	拆解费用
1	10%	4	2	10%	18	3	10%	6	3	56	6	5
2	20%	4	2	20%	18	3	20%	6	3	56	6	5
3	10%	4	2	10%	18	3	10%	6	3	56	30	5
4	20%	4	1	20%	18	1	20%	6	2	56	30	5
5	10%	4	8	20%	18	1	10%	6	2	56	10	5
6	5%	4	2	5%	18	3	5%	6	3	56	10	40

问题三：图 1 是 2 道工序、8 个零配件组装成成品的过程，零配件、半成品及成品的次品率如表 2 所示，请为生产的各个阶段做出决策，并说明决策的依据和对应的

指标结果。

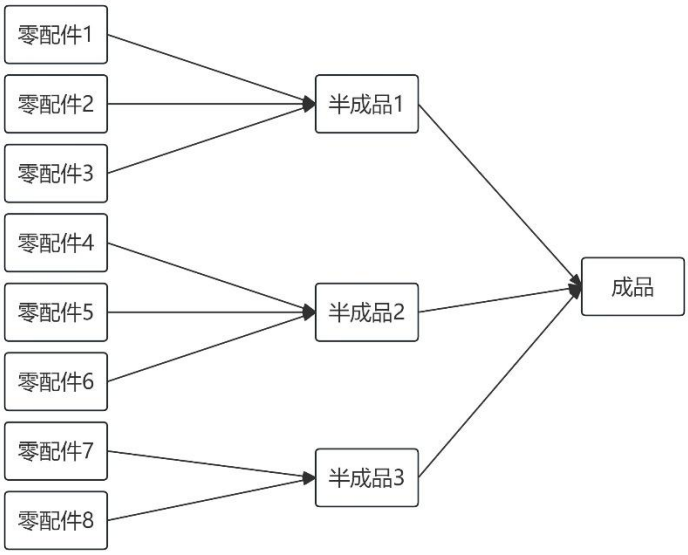


图 1 2 道工序、8 个零配件的组装情况

表 2 企业在生产中遇到的情况（问题三）

	次品率	购买单价	检测成本	装配成本	拆解费用	市场售价	调换损失
零配件 1	10%	2	1				
零配件 2	10%	8	1				
零配件 3	10%	12	2				
零配件 4	10%	2	1				
零配件 5	10%	8	1				
零配件 6	10%	12	2				
零配件 7	10%	8	1				
零配件 8	10%	12	2				
半成品 1	10%	4	4	8	6		
半成品 2	10%	4	4	8	6		
半成品 3	10%	4	4	8	6		
成品	10%	6	6	8	10	200	40

问题四：若问题 2 和问题 3 中零配件、半成品及成品的次品率都是通过问题 1 中的抽样检测方法得到的，请为问题 2 和问题 3 生产的各个阶段重新做出决策，并说明决策的依据和对应的指标结果。

二、 问题分析

2.1 问题一的分析

问题一需要设计出尽可能少的抽样次数使得假设的两种情况成立。为了判断这批

零配件是否可以接收，需对次品率p进行假设检验：原假设$H_0: p \leqslant p_0$（次品率不超过标称值），备择假设$H_1: p > p_0$（次品率超过标称值）。而后因为产品的合格与否只有两种情况，所以本文考虑采用二项分布来设计合理的抽样检测方案，当样本容量足够大时，可以根据独立同分布的中心极限定理，用正态分布去近似二项分布的概率质量函数，最后通过计算检验统计量、拒绝域等可以求出样本容量。

2.2 问题二的分析

问题二要根据 6 种具体情况为企业生产的各个阶段做出决策，并说明决策的依据和对应的指标结果。

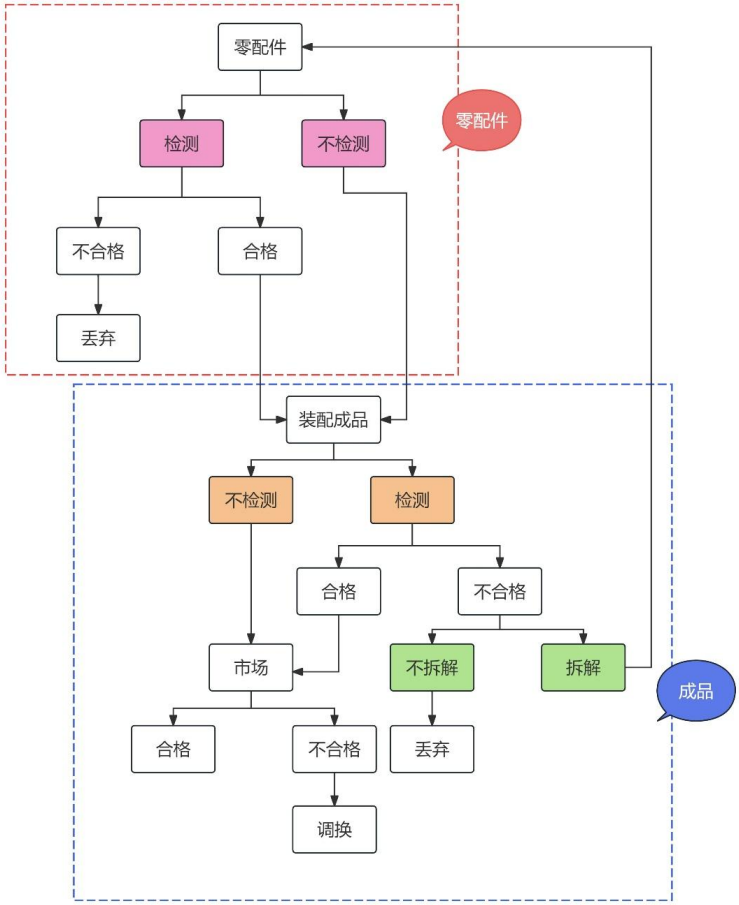


图 2 产品生产流程图

企业生产过程的各个阶段如图 2 所示，可以大致分为两个大板块（零配件、成品）和三个小板块（零配件是否检测，成品是否检测，不合格品是否拆解），因此本问有四个自变量：零配件 1 是否检测，零配件 2 是否检测，成品是否检测，不合格品是否检测。而后将信息进行量化，用 0 代表零配件 1(零配件 2)不接受检测，用 1 代表零配

件 1(零配件 2)接受检测，同理用 0 和 1 分别代表成品不接受检测和接受检测以及不合格成品不拆解和拆解。不合格成品共有两种来源，一是检测成品时发现的不合格成品，二是成品流入市场后返还的不合格成品。为了简化流程和模型，本文假设调换的成品是由公司仓库提供且被检测为正品，因而调换之后不会计入利润模型。题目中共列举 6 种情况，针对每一种情况，4 个自变量都可以构成 16 种方案，然后根据题目中所给参数，依次算出每种方案的购买成本、检测成本、装配成本、拆解成本、调换成本、销售收入，进而可以得到每种方案的利润。拆解出的零配件再次装配为成品时可能还是不合格品，此时会涉及多次拆解的问题，故本问对拆解次数也进行分析，模拟求出多次拆解时的利润。最后排序比较即可得最大利润及其对应的方案。

2.3 问题三的分析

问题三要 2 道工序、8 个零配件组装成成品的过程，重新为生产的各个阶段做出决策，并说明决策的依据和对应的指标结果。相较于问题二，问题三只有一种情况，但有两次组装的过程。由于不同数量的零配件和不同数量的半成品会对后续研究产生较大的影响，本问假设购买的零配件数量相同，以检测过且合成同一半成品中最少的零配件数量为该半成品的数量，其他处理情况与流程分析同问题二。在决策方案的选择过程中，本问使用模拟退火算法，通过多次迭代以及降温处理最后得到最大利润值和决策方案。

2.4 问题四的分析

问题四中，零配件 1、2，半成品和成品的次品率是通过抽样检测方法估计得到的随机变量。在问题二和问题三中，次品率为设定为确定值，而在问题四中，我们将次品率看作在一定信度下在置信区间中取值的变量，所以我们不能直接使用确定值来计算利润，要结合利润最大化和风险最小化进行决策。我们基于抽样检测方法得到的次品率的置信区间来量化其范围，分别将该置信区间的上限看作最坏情况，将置信区间下限看作最优情况，分别在两种情况下讨论决策。

三、 模型假设

1.用户退回不合格品调换得到的产品是经过成品检测合格的产品：为了维持企业的信誉和给用户更好的体验，在用户申请调换后再次拿到的产品一定是经过成品检测且检测结果为合格的产品，该调换的产品来自库存，因此不计入利润空间。

2.流入市场的不合格品用户都会选择调换：用户在使用不合格产品后会选择调换或直接丢弃，但本文为了得到更加精确的次品率，故假设流入市场的不合格品用户都会选择调换。

3.不合格成品中拆解出未检测过的零配件，则该零配件再次使用时一定会检测：因为成品不合格而导致被拆解成零配件，若没有经过检测直接再次使用，该零配件为不合格品的概率大大增加，因此不合格品中拆解出未检测过的零配件再次使用时一定要检测。

四、 符号说明

符号	说明
n	样本总数
p	次品率
X	样本中的次品数量
p_{real}	样本中的次品率
α	显著性水平
buy	零配件的购买成本
$test$	检测成本
$assem$	装配成本
$disass$	拆解成本
$return$	调换成本
$sales$	市场售价

五、 模型的建立与求解

5.1 问题一的模型建立与求解

根据题意将问题分为两个小问，第一小问在 95%的信度下认定零配件次品率超过标称值时拒收这批零配件，设计检测次数尽可能少的抽样检测方案；第二小问在 90%的信度下认定零配件次品率不超过标称值时接收这批零配件，设计检测次数尽可能少的抽样检测方案。

5.1.1 问题一（1）的求解

1.假设检验

原假设$H_0: p \leqslant p_0$，备择假设$H_1: p > p_0$。

其中p表示零配件的次品率，p_0表示标称值，其值为10%。

2.二项分布

零件是否为次品有两种结果，因此本文考虑用二项分布[1]的近似模型来设计合理的抽样检测方案。二项分布的概率质量函数为

$$P(X=k)=C_n^k p^k (1-p)^{n-k}, \tag{1}$$

其中n是样本总数，p是次品率，k是次品数量。

$$X=np_{real}, \tag{2}$$

其中X是样本中的次品数量，p_{real}是样本中的次品率。

3.正态分布近似二项分布

当n足够大时，可以根据独立同分布的中心极限定理[2]，用正态分布对二项分布进行近似

$$X \sim N(np, np(1-p)). \tag{3}$$

4.检验统计量

根据统计检验，计算检验统计量的观察值

$$z=\frac{p_{real}-p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}, \tag{4}$$

其中z是标准正态分布的检验统计量，。

5.拒绝域

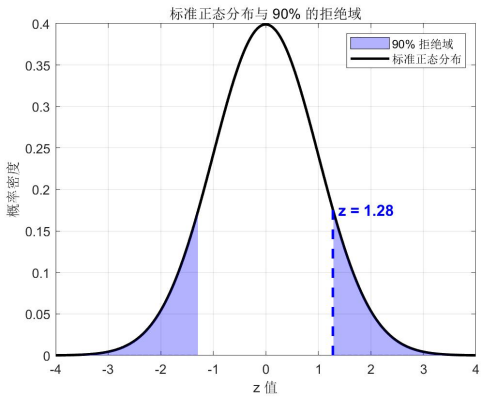


图 3 标准正态分布与 90%信度的拒绝域

在 95%的信度下认定 $p > p_0$，即显著性水平 α 为 0.05

$$\Phi(z_\alpha) = 1 - \alpha = 0.95, \tag{5}$$

查询标准正态分布表可得 z_α 的值为 1.645，则 H_0 的拒绝域为 $z \geqslant z_\alpha = 1.645$ 。

6.求解样本数量 n

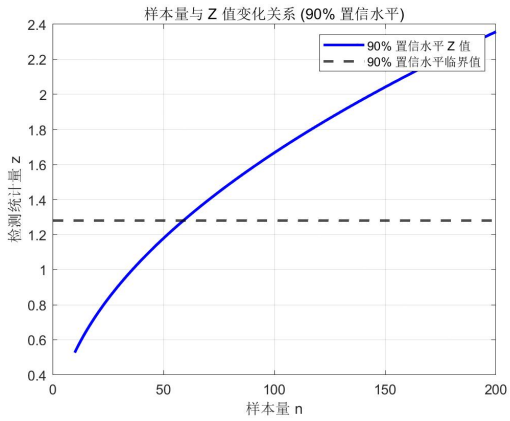


图 4 样本量与 z 值的变化关系（信度 90%）

由 H_0 的拒绝域 $z \geqslant z_\alpha$ 和检验统计量 z 的值可以得到

$$n \geqslant \frac{z_\alpha^2 p_0 (1 - p_0)}{(p_{real} - p_0)^2}, \tag{6}$$

实际次品率与标称值之间存在一定的误差，假定该该误差为 0.05，则可以算出**样本数量 n 的值为 98**。

5.1.2 问题一（2）的求解

1.假设检验

原假设$H_0: p \geqslant p_0$，备择假设$H_1: p < p_0$。

其中p表示零配件的次品率，p_0表示标称值，其值为 10%。

2.二项分布

零件是否为次品有两种结果，因此本文考虑用二项分布的近似模型来设计合理的抽样检测方案。二项分布的概率质量函数为

$$P(X=k)=C_n^k p^k (1-p)^{n-k}, \tag{7}$$

其中n是样本总数，p是次品率，k是次品数量。

$$X = np_{real}, \tag{8}$$

其中X是样本中的次品数量，p_{real}是样本中的次品率。

3.正态分布近似二项分布

当n足够大时，可以根据独立同分布的中心极限定理，用正态分布对二项分布进行近似

$$X \sim N(np, np(1-p)). \tag{9}$$

4.检验统计量

根据统计检验，计算检验统计量的观察值

$$z = \frac{p_{real} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}, \tag{10}$$

其中z是标准正态分布的检验统计量。

5.拒绝域

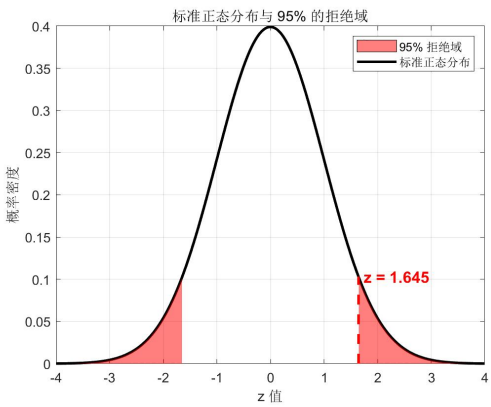


图 5 标准正态分布与信度 95%的拒绝域

在 90%的信度下认定$p \leqslant p_0$，即显著性水平α为 0.1

$$\Phi(z_\alpha) = 1 - \alpha = 0.9, \tag{11}$$

查询标准正态分布表可得z_α的值为 1.28，则H_0的拒绝域为$z \leqslant -z_\alpha = -1.28$。

6.求解样本数量n

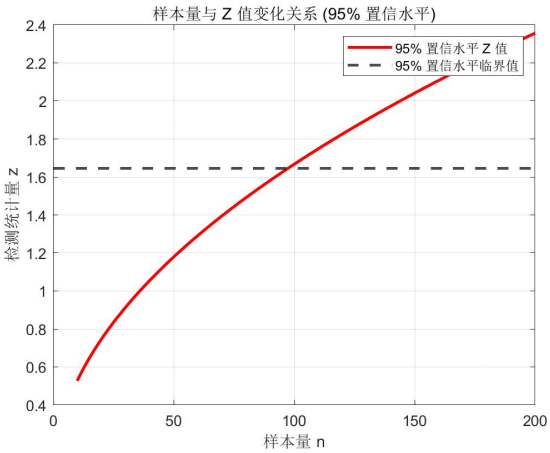


图 6 样本量与 z 值的变化关系（信度 95%）

由H_0的拒绝域$z \leqslant -z_\alpha$和检验统计量z的值可以得到

$$n \geqslant \frac{z_\alpha^2 p_0 (1 - p_0)}{(p_{real} - p_0)^2}, \tag{12}$$

求得**样本数量n的值为 59**。

5.2 问题二模型的建立与求解

问题二要针对 6 种具体情况为企业生产的各个阶段做出决策，并说明决策的依据和对应的指标结果。

5.2.1 问题二模型的建立

1.购买成本

$$C_{buy} = n_1 \cdot buy_1 + n_2 \cdot buy_2, \tag{13}$$

其中n_1表示零配件 1 的数量（默认为 100），buy_1表示零配件 1 的购买成本，n_2表示零配件 2 的数量（默认为 100），buy_2表示零配件 2 的购买成本，C_{buy}表示总购买成本。

2.检测成本

$$C_{test}=x_1\cdot n_1\cdot test_1+x_2\cdot n_2\cdot test_2+x_3\cdot n_3\cdot test_3, \tag{14}$$

其中x_1表示是否检测零配件 1（不检测x_1的值为 0，检测x_1的值为 1），$test_1$表示零配件 1 的检测成本，x_2表示是否检测零配件 2（不检测x_2的值为 0，检测x_2的值为 1），$test_2$表示零配件 2 的检测成本，x_3表示是否检测成品（不检测x_3的值为 0，检测x_3的值为 1），$test_3$表示成品的检测成本，n_3表示成品的数量，C_{test}表示总检测成本。

3.装配成本

$$C_{assem}=n_3\cdot assem, \tag{15}$$

其中$assem$表示装配成本，C_{assem}表示总装配成本。

4.拆解成本

$$C_{disass}=N_{fail}\cdot disass\cdot x_4, \tag{16}$$

其中N_{fail}表示不合格产品的数量，$disass$表示拆解成本，x_4表示是否拆解不合格成品（不拆解x_4的值为 0，拆解x_4的值为 1），C_{disass}表示总拆解成本。

5.调换成本

$$C_{return}=N_{return}\cdot return, \tag{17}$$

其中N_{return}表示调换数量，$return$表示调换成本，C_{return}表示总调换成本。

6.销售收入

$$R_{sales}=N_{sales}\cdot sales, \tag{18}$$

其中N_{sales}表示销售数量，$sales$表示市场售价，R_{sales}表示总销售收入。

7.总利润

$$R_{tatol}=R_{sales}-C_{buy}-C_{test}-C_{assem}-C_{disass}-C_{return}. \tag{19}$$

其中R_{total}表示总利润。

成品的数量n_3，不合格产品的数量N_{fail}，调换数量N_{return}，销售数量N_{sales}的大小会随x_1，x_2，x_3，x_4的变化而变化，其余变量均由表 1 中的 6 种情况确定。

5.2.2 问题二模型的求解

1.信息量化

针对不同情况，从表 1 中得到对应情况的buy_1，buy_2，$test_1$，$test_2$，$test_3$，$assem$，$return$，$sales$，$disass$的值，假设n_1、n_2的值均为 100，x_1，x_2，x_3，x_4的值均为 0 或 1。

2.判断是否检测零配件 1

计算出零配件 1 的成本

$$C_1 = n_1 \cdot buy_1 + x_1 \cdot n_1 \cdot test_1. \tag{20}$$

①若x_1的值为 1，则表示要检测零配件 1，检测后有部分不合格的零配件 1 被丢弃，此时的零配件数量为$(1-p_1)n_1$，其中p_1为零配件 1 的次品率，而零配件 1 中的正品率P_1为 1。

②若x_1的值为 0，则表示不检测零配件 1，不检测代表没有零配件 1 被丢弃，此时的零配件 1 的数量仍为n_1，而零配件 1 中的正品率P_1为$1-p_1$。

3.判断是否检测零配件 2

计算出零配件 2 的成本

$$C_2 = n_2 \cdot buy_2 + x_2 \cdot n_2 \cdot test_2. \tag{21}$$

①若x_2的值为 1，则表示要检测零配件 2，检测后有部分不合格的零配件 2 被丢弃，此时的零配件 2 数量为$(1-p_2)n_2$，其中p_2为零配件 2 的次品率，而零配件 2 中的正品率P_2为 1。

②若x_2的值为 0，则表示不检测零配件 2，不检测代表没有零配件 2 被丢弃，此时的零配件 2 的数量仍为n_2，而零配件 2 中的正品率P_2为$1-p_2$。

4.判断是否检测成品

计算出成品的成本

$$C_3 = n_3 \cdot assem + x_3 \cdot n_3 \cdot test_3, \tag{22}$$

其中n_3的值等于零配件 1 数量和零配件 2 数量中较小的值。

①若x_3的值为 1，则表示要检测成品，检测意味着部分不合格的成品被丢弃，成品的最终次品率$P_{real}=1-P_1\cdot P_2\cdot p_3$,其中$p_3$表示成品的次品率，此时可销售的合个成品数量为$(1-P_{real})\cdot n_3$，不合格的成品有$P_{real}\cdot n_3$个未流入市场。

②若x_3的值为 0，则表示不检测成品，不检测即没有成品被丢弃，成品的最终次品率$P_{real}=1-P_1\cdot P_2$,此时可销售的成品数量为$(1-P_{real})\cdot n_3$，不合格的成品有$P_{real}\cdot n_3$个未流入市场。

5.判断不合格品是否拆解

方法一：探究检测成本、装配成本、购买成本、销售额、拆解成本之间的关系。

①拆解

因为成品不合格而导致被拆解成零配件，若没有经过检测直接再次使用，该零配件为不合格品的概率大大增加，因此本文假设不合格品若选择拆解，则拆解出未检测过的零配件再次使用时一定要检测，根据不同情况有

$$C_{again}=\begin{cases} test_1\cdot P_{real}\cdot n_3,\ [0,1] \\ test_2\cdot P_{real}\cdot n_3,\ [1,0] \\ (test_1+test_2)\cdot P_{real}\cdot n_3,\ [0,0] \\ 0,\ [1,1] \end{cases}, \tag{23}$$

其中C_{again}表示拆解出未检测过的零配件的检测成本，$[0,0]$表示零配件 1 未检测，零配件 2 未检测，其余同理。

不合格成品数量

$$Y=\begin{cases} n_3\cdot P_{real},\ [1,1] \\ n_3\cdot P_{real}\cdot(1-p_1),\ [0,1] \\ n_3\cdot P_{real}\cdot(1-p_2),\ [1,0] \\ \min\{n_3P_{real}(1-p_1),n_3P_{real}(1-p_2)\},\ [0,0] \end{cases}, \tag{24}$$

其中[0,1]表示零配件 1 未检测，零配件 2 已经检测过，其余同理。

$$\begin{aligned} f_1(k)&=sales\cdot(1-P_{real})\cdot Y+sales\cdot(1-P_{real})\cdot P_{real} \\ &\quad+\cdots+sales\cdot {P_{real}}^{k-1}\cdot Y\cdot(1-P_{real}) \\ &=sales\cdot Y\cdot(1-P_{real})\cdot\frac{1-{P_{real}}^k}{1-P_{real}}, \end{aligned} \tag{25}$$

其中$f_1(k)$为不合格成品经过拆解组装后的累计销售额，k为拆解次数。

$$\begin{aligned} f_2(k) = = & (C_{assem} + c_{test}) \cdot Y + (C_{assem} + c_{test}) \cdot Y \cdot P_{real} \\ & + \cdots + (C_{assem} + C_{test}) \cdot Y \cdot {P_{real}}^k \\ & = (C_{assem} + C_{test}) \cdot Y \cdot \frac{1 - {P_{real}}^k}{1 - P_{real}}, \end{aligned} \tag{26}$$

其中$f_2(k)$为不合格成品经过拆解后需要装配和检测的累计费用。

$$\begin{aligned} f_3(k) = & C_{disass} \cdot Y \cdot P_{real} + C_{disass} \cdot Y \cdot {P_{real}}^2 \\ & + \cdots + C_{disass} \cdot Y \cdot {P_{real}}^{k-1} \\ = & C_{disass} \cdot Y \cdot \frac{P_{real}(1 - {P_{real}}^{k-1})}{1 - P_{real}}, \end{aligned} \tag{27}$$

其中$f_3(k)$为不合格成品拆解的累计费用，第一次拆解费用与$f_3(k)$计算规则不同但有固定值

$$C_{disass1} = disass \cdot n_3 \cdot P_{real}. \tag{28}$$

综上所述，得到除去第一次拆解费用的不合格成品拆解利润函数为

$$f(k) = f_1(k) - f_2(k) - f_3(k), \tag{29}$$

$$f(k) = f_1(k) - f_2(k) - f_3(k), \tag{30}$$

再由利润公式可得

$$R_{total} = R_{sales} - C_1 - C_2 - C_3 - C_{again} - C_{disass} + f(k). \tag{31}$$

②不拆解

如果选择不拆解，则不合格成品直接丢弃，不再产生相应利润

$$R_{total} = R_{sales} - C_1 - C_2 - C_3. \tag{32}$$

方法二：穷尽可拆解的成品直至零配件 1、2 数量不再匹配时，停止拆解次。

6.枚举法结合模拟退火算法思想求得结果

根据模拟退火算法的思想，解空间可以随机生成并改变，在一定程度上接收错误的新解，随后进行迭代与降温得到每种情况下的最优发难和最大利润，但由于本问的解空间仅有 16 种元素，为了避免算法自身的缺点导致只能找到局部最优解，因此本问用枚举法得到解空间，并逐一计算比较排序后得到最大利润和决策方案。

5.2.3 问题二模型的结果

方法一的结果

四个自变量x_1，x_2，x_3，x_4的取值均为 2 种（0 或 1），意味着在每一种情况下都会有 16 种结果的产生，因此本问用枚举法，利用 Matlab 程序遍历，将每种情况下的 16 种结果一一验算，以此发现总成本最小的决策方案。篇幅受限，此处仅列举情况 1 的结果，详见附录 1。

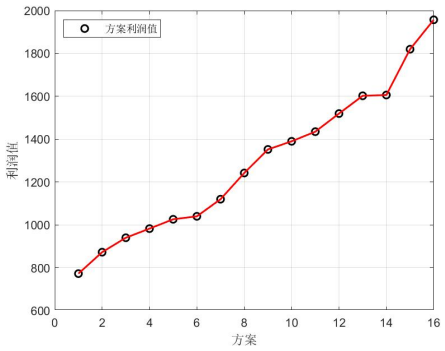


图 7 情况 1 方案 1 利润变化图

计算出情况 1 中 16 种方案的利润升序排列后画出图 7，横坐标是方案的序号，纵坐标是对应方案的利润值，因为对利润进行升序排列，故图像呈现出单调递增的趋势，最低利润不足 800 元，最高利润接近 2000 元。

成品不合格而导致被拆解成零配件，若这些零配件没有经过检测直接再次使用，该零配件装配成的成品是不合格品的概率大大增加，因此本文规定，不合格品中拆解出未检测过的零配件再次使用时一定要检测，零配件检测结果为合格再继续使用，但合格的零配件装配成的成品也有一定概率是不合格品，此时会涉及再次拆解的问题，故本问对拆解次数也进行分析，模拟求出多次拆解时的利润。

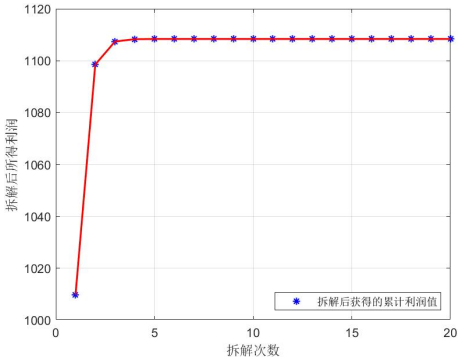


图 8 拆解次数与利润的关系图

从图 8 中可以发现，当零配件拆解的次数达到 4 次以后，随着次数的增加而利润值几乎不变，因此为了节省人力物力，若零配件经过 4 次拆解后装配成的成品仍为不

合格品，则直接将该不合格成品丢弃，不再考虑其是否拆解的问题。

情况 1 决策方案为 0-0-0-1，即零配件 1 不检测-零配件 2 不检测-成品不检测-不合格品拆解，且每一个零配件经历的拆解次数最多为 4 次，此时最大利润为 1957。

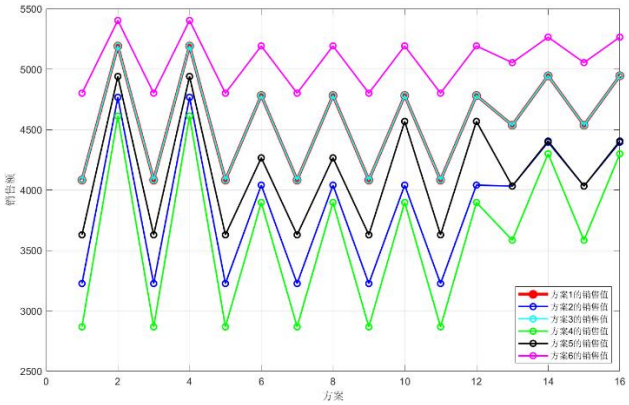


图 9 6 种情况 16 种方案的销售额

观察图 9 可以发现，不同情况下，都是方案 2 和方案 4 的销售额较大，说明这两种方案对于决策方案，因此企业在做决策时可以优先考虑这两种方案。

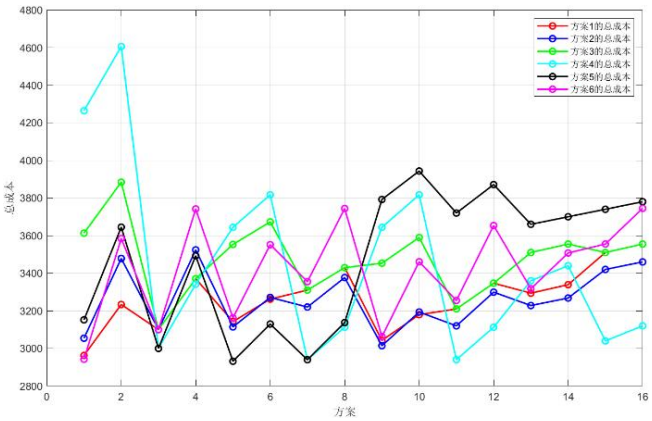


图 10 6 种情况 16 种方案的总成本

针对每种情况算出不同方案的销售额和总成本，二者相减即可得到最大利润，该方案也为决策方案。

表 3 问题二决策方案

情况	零件 1 是否检测	零件 2 是否检测	成品是否检测	成品是否拆解	利润	拆解次数
1	不检测	不检测	不检测	拆解	1957	4
2	不检测	不检测	不检测	拆解	1289	4
3	不检测	不检测	检测	拆解	1820	4
4	不检测	不检测	检测	拆解	1272	5
5	不检测	不检测	检测	拆解	1447	4
6	不检测	不检测	不检测	不拆解	1859	4

从表 3 中可以看出，各种情况下都偏向于零配件 1 和零配件 2 不检测，不合格品

拆解且拆解次数为 4 次时为决策方案，这与零配件的检测成本和不合格品的拆解成本有关，拆解成本较低就倾向于不检测但拆解，拆解成本较高就倾向于检测但不拆解，符合企业要求利润最大化的需求。

方法二的结果：

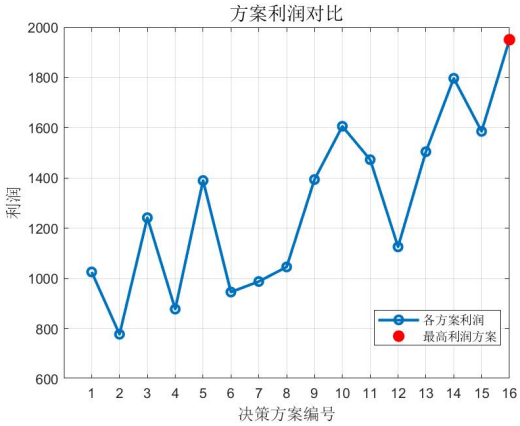


图 11 情况 1 各方案利润变化图

从图中可以看出，不同方案之间利润差距较大，最低利润仅 800，最高利润接近 2000，因此企业需要选择最优决策方案以获取更多利润。

表 4 问题二方法二决策方案表

情况	零件 1 是否检测	零件 2 是否检测	成品是否检测	成品是否拆解	利润
1	不检测	不检测	不检测	拆解	1950.7
2	检测	检测	不检测	拆解	924
3	不检测	不检测	检测	拆解	1797
4	检测	检测	检测	拆解	1180
5	检测	检测	检测	拆解	1180
6	不检测	不检测	不检测	不拆解	1876

根据方法一的结果表和方法二的结果表对比，两种方法的决策方案相近、最大利润相近，两种方法相互印证，证明了模型的稳定性和准确性。但方法二的结果与零配件初始量n_1和n_2相关，相对误差较大，方法一不受零配件购买量的影响，因此方法一比方法二更加科学合理。

5.3 问题三模型的建立与求解

5.3.1 问题三模型的建立

1.零配件

（1）购买成本

$$A_{buy}=\sum_{i=1}^{8} n_i \cdot buy_i, \tag{33}$$

其中n_i表示零配件i的数量，buy_i表示零配件i的购买成本，$i=1,2,\cdots,8$表示零配件1到零配件8，A_{buy}表示零配件的总购买成本。

（2）检测成本

$$A_{test}=\sum_{i=1}^{8} x_i \cdot n_i \cdot test_i, \tag{34}$$

其中x_i表示是否检测零配件i，$test_i$表示零配件i的检测成本，A_{test}表示零配件的总检测成本。

2.半成品

（1）装配成本

$$B_{assem}=\sum_{i=9}^{11} n_i \cdot assem_i, \tag{35}$$

其中n_i表示半成品i的数量，$assem_i$表示半成品i的装配成本，$i=9,10,11$表示半成品1到半成品3，B_{assem}表示半成品的总装配成本。

（2）检测成本

$$B_{test}=\sum_{i=9}^{11} x_i \cdot n_i \cdot test_i, \tag{36}$$

其中x_i表示半成品i是否检测，$test_i$表示半成品i的检测成本，B_{test}表示半成品的总装配成本。

（3）拆解成本

$$B_{disass}=\sum_{i=9}^{11} x_i \cdot disass_i \cdot B_{fail\,i}, \tag{37}$$

其中x_i表示半成品i是否检测，$test_i$表示半成品i的检测成本，B_{fail_i}表示不合格的半成品数量，表示B_{disass}的半成品的总拆解成本。

3.成品

（1）装配成本

$$C_{assem} = n \cdot assem, \tag{38}$$

其中n表示成品的数量，$assem$表示成品的装配成本，C_{assem}表示成品的总装配成本。

（2）检测成本

$$C_{test} = x \cdot n \cdot test, \tag{39}$$

其中x表示成品是否检测，$test$表示成品的检测成本，C_{test}表示成品的总检测成本。

（3）拆解成本

$$C_{disass} = disass \cdot x \cdot N_{fail}, \tag{40}$$

其中$disass$表示成品的拆解成本，N_{fail}表示不合格的成品数量，C_{disass}表示成品的总拆解成本。

（4）调换成本

$$C_{return} = return \cdot N_{return}, \tag{41}$$

其中$return$表示成品的调换成本，N_{return}表示需要调换的成品数量，C_{return}表示成品的总调换成本。

（5）销售收入

$$R_{sales} = sales \cdot N_{sales}, \tag{42}$$

其中$sales$表示成品的市场售价，N_{sales}表示成品的销售数量，R_{sales}表示成品的总销售收入。

4.总利润

$$\begin{aligned} R_{total} = R_{sales} - A_{buy} - A_{test} - B_{test} - B_{assem} - B_{disass} - C_{assem} \\ - C_{test} - C_{disass} - C_{return.} \end{aligned} \tag{43}$$

其中R_{total}表示总利润。

5.3.2 问题三模型的求解

算法：求解目标函数采用的模拟退火算法[3]如下。

（1）解空间

解空间S零配件1-8是否检测$x_i(i=1,2,\cdots,8)$，半成品1-3是否检测$x_i(i=9,10,11)$，成品x是否检测，半成品是否拆解，成品是否拆解，随机生成初始

解。

（2）目标函数

此时的目标函数为最大化企业的利润，即

$$\begin{aligned}\max\ R_{total}=R_{sales}-A_{buy}-A_{test}-B_{test}-B_{assem}-B_{disass}\\-C_{assem}-C_{test}-C_{disass}-C_{return.}\end{aligned}\tag{44}$$

（3）新解的产生

1、交换法

任选序号$u,v(u<v)$交换u和v之间的顺序，此时的新路径为：

$$e_1\cdots e_{u-1}e_v e_{u+1}\cdots e_5\tag{45}$$

2、位移法

任选序号u，v和$w\,(u<v<w)$，将u，v之间的路径插到w之后，此时的新路径为：

$$e_1\cdots e_{u-1}e_v e_{u+1}\cdots e_5\tag{46}$$

3、倒置法

任选序号u，v，如果u大于v，则将u和v之间（包括u，v）的序号倒置，此时的新路径为：

$$e_1\cdots e_{u-1}e_{v+1\cdots}e_w e_u\cdots e_v e_{w+1}\cdots e_5\tag{47}$$

随机生成一个 0 到 1 之间的随机数s，若$0<s<0.33333$，则选择交换法产生新解，若$0.3333<s<0.6666$。则选择位移法产生新解，其余则选择倒置法产生新解。

（4）代价函数差

路径差Δf是后一条路径与前一条路径长度之差，可表示为：

$$\Delta f=z_{u+1}-z_u\tag{48}$$

（5）接受准则

$$p=\begin{cases}1, & \Delta f<0\\ \exp(-\Delta f/T), & \Delta f\geqslant 0\end{cases}\tag{49}$$

其中，本问设定的初始温度T为 1000。如果$\Delta f<0$，则接受新的路径；否则以

概率$\exp(-\Delta f/T)$接受新的路径，即当$\exp(-\Delta f/T)$的值大于 0 到 1 的随机数则接受。

（6）降温

利用选定的降维系数$\alpha=0.95$进行降温，即$T\leftarrow\alpha T$，得到新的温度。

（7）终止条件

用迭代次数作为终止条件，判断退火过程是否结束，若迭代次数大于等于 5000，算法结束，退出当前状态。

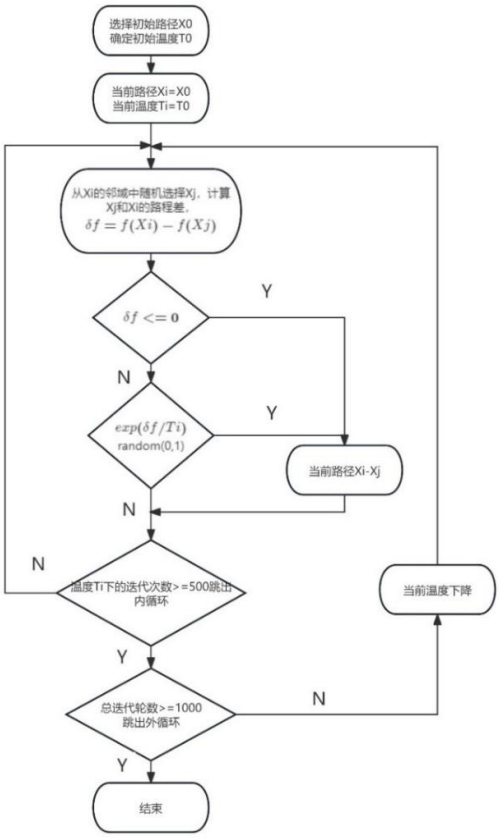


图 12 模拟退火算法流程图

5.3.3 问题三模型的结果

12 个自变量的取值均为 2 种（0 或 1），意味着会有 2^8 种结果的产生，因此本问用模拟退火算法发现总成本最小的决策方案。

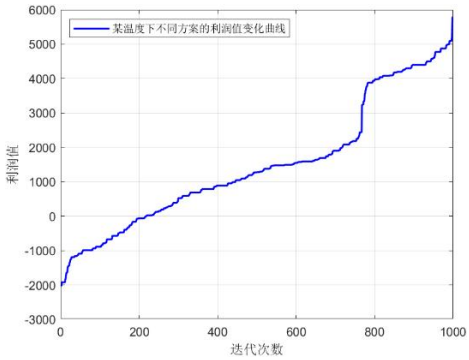


图 13 利润值随迭代次数的变化

从图 13 可以出看出，随着迭代次数的增加，利润值也在增加，并且在所规定的迭代终止次数 5000 次以下，仅需 1000 次即可得到最大利润值。

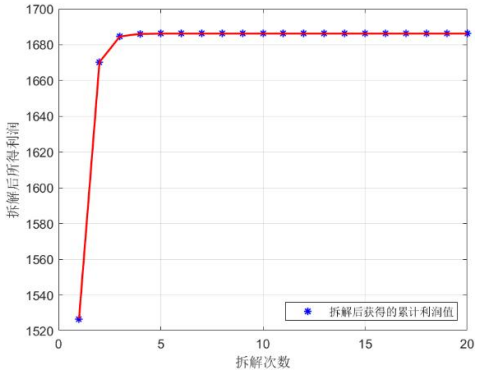


图 14 拆解次数与利润的关系图

从图 14 中可以发现，当零配件拆解的次数达到 5 次以后，随着次数的增加而利润值几乎不变，因此为了节省人力物力，若零配件经过 5 次拆解后装配成的成品仍为不合格品，则直接将该不合格成品丢弃，不再考虑其是否拆解的问题。

表 5 问题三决策方案

方案	决策	方案	决策
零配件 1 是否检测	不检测	半成品 1 是否检测	不检测
零配件 2 是否检测	不检测	半成品 2 是否检测	检测
零配件 3 是否检测	不检测	半成品 3 是否检测	不检测
零配件 4 是否检测	不检测	半成品 1 是否拆解	拆解
零配件 5 是否检测	不检测	半成品 2 是否拆解	不拆解
零配件 6 是否检测	不检测	半成品 3 是否拆解	拆解
零配件 7 是否检测	不检测	成品是否检测	检测
零配件 8 是否检测	不检测	成品是否拆解	拆解

决策方案为 0-0-0-0-0-0-0-0-0-0-1-0-1-0-1-1-1，**即零配件 1 不检测-零配件 2 不检测-零配件 3 不检测-零配件 4 不检测-零配件 5 不检测-零配件 6 不检测-零配件 7 不检测-零配件 8 不检测-半成品 1 成品不检测-半成品 2 检测-半成品 3 不检测-半成品 1 拆解-**

半成品 2 不拆解-半成品 3 拆解-成品检测-不合格品拆解，且每一个零配件经历的拆解次数最多为 5 次，此时最大利润为 6352。

5.4 问题四模型的建立与求解

问题 2 和问题 3 中零配件、半成品及成品的次品率都是通过问题 1 中的抽样检测方法得到的，为问题 2 和问题 3 生产的各个阶段重新做出决策，并说明决策的依据和对应的指标结果。

5.4.1 问题四模型的建立

延续问题一的抽样思想，将零配件 1、2，半成品和成品的次品数量的二项分布，均根据中心极限定理近似为正态分布。

1.点估计

$$\hat{p}=\frac{X}{n}, \tag{50}$$

其中$\hat{p}$为次品率的点估计量。

2.置信区间

对于置信水平$1-\alpha$，次品率的置信区间为

$$\left(\hat{p}-z_{\frac{\alpha}{2}}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}},\hat{p}+z_{\frac{\alpha}{2}}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}\right). \tag{51}$$

3.抽样方案

此处给出表 6 中的 4 种抽样假设进行分析。

表 6 置信区间表

估计次品率	允许的误差	置信水平	区间估计	抽样数	抽样决策
7%	2%	80%	[0.0444, 0.0825]	267	不检测
7%	2%	90%	[0.0478, 0.0950]	322	不检测
10%	2%	99%	[0.0804, 0.1205]	1493	检测
15%	2%	90%	[0.1273, 0.1670]	826	检测

分析表 6 可以看出，在企业接受次品率标称值为 10%的情况下，估计次品率相同时，置信水平越大，置信区间估计值的上下限越大，区间长度越大，说明次品率在该区间内的可靠性更高，所需抽样数越多。当估计次品率小于标称值时基本建议检测，在估计次品率大于标称值时，无论置信水平高低和抽样数多少，基本建议不检测。并且置信水平与所需的抽样数呈正相关。

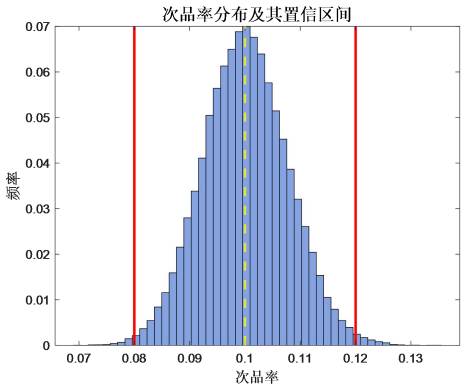


图 15 次品率分布及其置信区间

图 15 是模拟 10000 次抽样的结果，从图中可以发现，在 99%的置信水平下，$z_{\alpha/2}$ 的值为 2.5698，次品率的置信区间为[0.0749, 0.11391]，意味着在 99%的置信水平下，本问认为真实的次品率有 99%的可能性位于 7.49%到 11.39%之间。

初次判断应该对零配件 1、零配件 2、半成品、成品进行检测。虽然置信区间表明次品率的上限高于企业的可接受范围（10%），但单纯依靠置信区间并不足以决定是否检测，还需要综合考虑检测成本等因素。

因此在原本次品率的基础上增加置信区间，动态调整次品率，使用置信区间的上限来计算最坏情况下的次品率，估算出最坏情况下的利润。使用置信区间的下限来计算最好情况下的次品率，估算出最好情况下的利润。

5.4.2 问题四（1）模型的结果

对问题二中的代码改进即可得到结果。

情况 1：当零配件 1 购买单价为 4，检测成本为 2；零配件 2 购买单价为 18，检测成本为 3；成品装配成本为 6，检测成本为 3；市场售价为 56；不合格成品调换损失为 6，拆解费用为 5 时的最佳决策方案有 4 种：

不检测零配件 1，不检测零配件 2，不检测成品，对不合格成品进行调换。

不检测零配件 1，要检测零配件 2，不检测成品，对不合格成品进行调换。

要检测零配件 1，不检测零配件 2，不检测成品，对不合格成品进行调换。

要检测零配件 1，不检测零配件 2，要检测成品，对不合格成品进行调换。

此时平均利润为：2967.6984，最坏情况利润为：3699.2301，最好情况利润为：2236.1667。

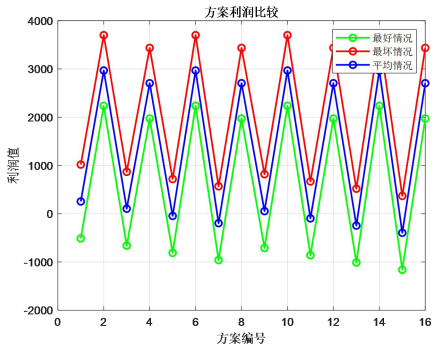


图 16 情况 1 不同方案利润的比较

观察图 16 可以发现，不同方案下，同一情况的变化趋势相同，说明方案对利润的影响程度较大。

表 7 问题四（1）决策方案

情况	方案	零件 1 是否检测	零件 2 是否检测	成品是否检测	成品是否拆解	利润
1	1	不检测	不检测	不检测	拆解	2967.698
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	不检测	检测	拆解	
2	1	不检测	不检测	不检测	拆解	2967.698
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	检测	不检测	不拆解	
3	1	不检测	不检测	不检测	拆解	3904.1
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	不检测	不检测	拆解	
4	1	不检测	不检测	不检测	拆解	2967.698
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	检测	不检测	拆解	
5	1	不检测	不检测	不检测	拆解	3432.993
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	检测	不检测	拆解	
6	1	不检测	不检测	不检测	拆解	3059.848
	2	不检测	检测	不检测	拆解	
	3	检测	不检测	不检测	拆解	
	4	检测	检测	不检测	拆解	

5.4.3 问题四（2）模型的结果

条件：当零配件购买单价为 2，检测成本为 1；零配件 2 购买单价为 8，检测成本

为 1；当零配件 3 购买单价为 12，检测成本为 2；零配件 4 购买单价为 2，检测成本为 1；当零配件 5 购买单价为 8，检测成本为 1；零配件 6 购买单价为 12，检测成本为 2；当零配件 7 购买单价为 8，检测成本为 1；零配件 8 购买单价为 12，检测成本为 2；半成品 1、2、3 的装配成本为 8，检测成本为 4，拆解费用为 6；成品的装配成本为 8，检测成本为 6，拆解费用为 10，市场售价为 200，调换损失为 40。

1.在最好情况下，最大利润为:9732。

表 8 问题四（2）决策方案

	是否检测			是否检测		是否拆解	
零配件 1	0	不检测	零配件 7	0	不检测		
零配件 2	0	不检测	零配件 8	0	不检测		
零配件 3	0	不检测	半成品 1	0	不检测	1	拆解
零配件 4	0	不检测	半成品 2	1	检测	1	拆解
零配件 5	0	不检测	半成品 3	0	不检测	1	拆解
零配件 6	0	不检测	成品	0	不检测	1	拆解
利润：9732							

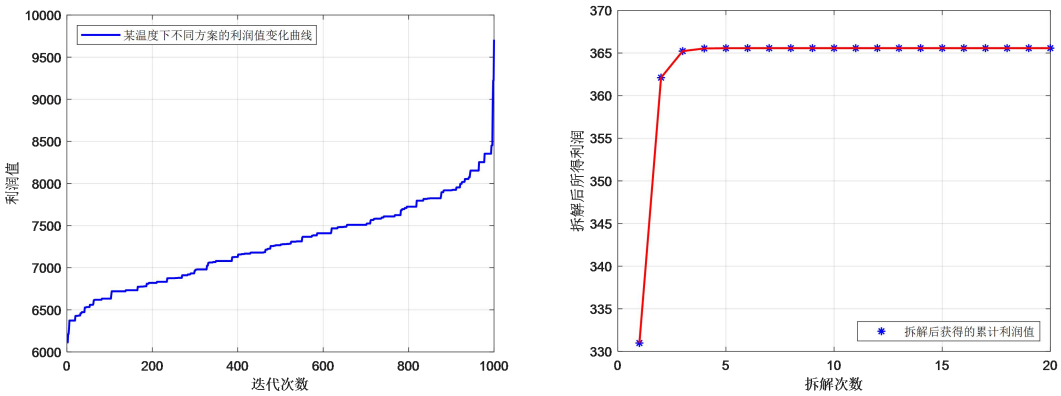


图 17 利润与迭代次数（左）和拆解次数（右）的关系

决策方案为:零配件 1 不检测，零配件 2 不检测，零配件 3 不检测，零配件 4 不检测，零配件 5 不检测，零配件 6 不检测，零配件 7 不检测，零配件 8 不检测，半成品 1 不检测，半成品 2 要检测，半成品 3 不检测，半成品 1 要拆解，半成品 2 要拆解，半成品 3 要拆解，成品不检测，成品要拆解。

2.在最坏情况下，最大利润为:8573。

表 9 问题四（2）的决策方案

是否检测			是否检测			是否拆解	
零配件 1	0	不检测	零配件 7	0	不检测		
零配件 2	0	不检测	零配件 8	0	不检测		
零配件 3	0	不检测	半成品 1	0	不检测	1	拆解
零配件 4	0	不检测	半成品 2	1	检测	0	不拆解
零配件 5	0	不检测	半成品 3	0	不检测	1	拆解
零配件 6	0	不检测	成品	1	检测	1	拆解
利润：8573							

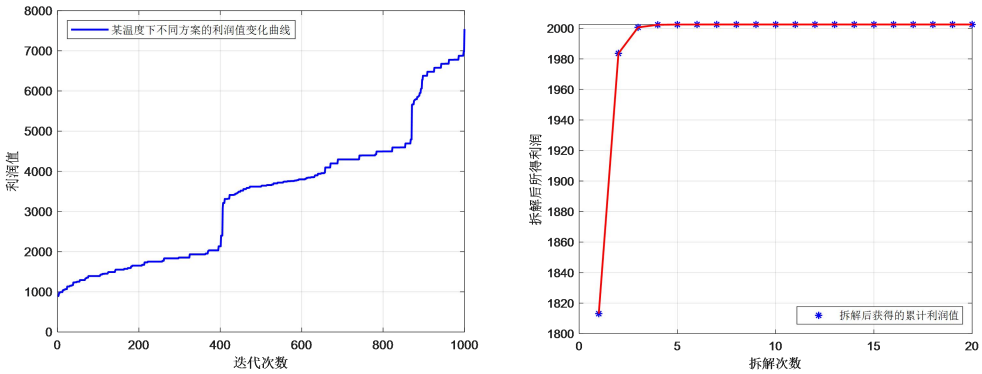


图 18 利润与迭代次数（左）和拆解次数（右）的关系

决策方案为:零配件 1 不检测，零配件 2 不检测，零配件 3 不检测，零配件 4 不检测，零配件 5 不检测，零配件 6 不检测，零配件 7 不检测，零配件 8 不检测，半成品 1 不检测，半成品 2 要检测，半成品 3 不检测，半成品 1 要拆解，半成品 2b 不拆解，半成品 3 要拆解，成品要检测，成品要拆解。

六、 模型的评价、改进与推广

6.1 模型的优点

1.充分考虑生产过程中的各个阶段可能涉及到的情况，问题二、问题三中就不合格成品是否拆解进行讨论，给出了两种不同情况下的计算公式。

2.问题二的求解用了枚举法一一算出每种方案的利润再进行比较，因为每种情况下只有 16 种方案，方案数过少，避免了使用优化算法时出现局部最优解的情况。

3.进行求解的代码是通用的，每次情况改变时只需要变动部分成本和利润的参数即可重新求解，便于企业进行多种零件多种生产过程的利润求解。

6.2 模型的缺点

针对性过强，如假设用户拿到不合格成品后都会选择调换，而实际生活中会有部分顾客直接将产品丢弃，因此本文的假设还不够合理，没有充分考虑实际情况。

6.3 模型的改进

将假设部分与实际贴合，构建更为合理的模型。

七、 参考文献

[1]孙慧静,马启建,王丽英.与二项分布有关的概率分布之间关系注记[J].高等数学研究,2023,26(04):1-5+105.

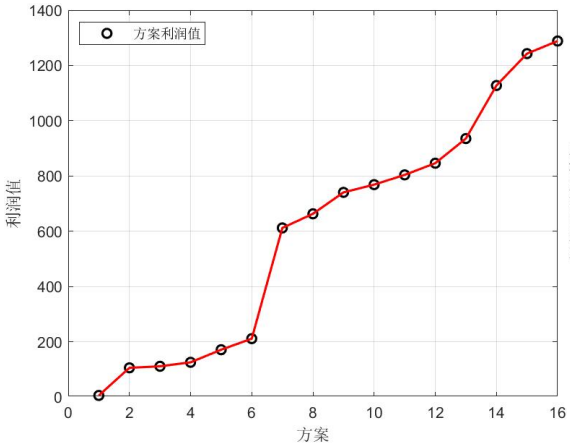
[2]李真.《概率论与数理统计》中定理的联系教学--以独立同分布的中心极限定理和样本均值的抽样分布定理为例[J].赤峰学院学报(自然科学版),2018,34(10):133-134.DOI:10.13398/j.cnki.issn1673-260x.2018.10.046.

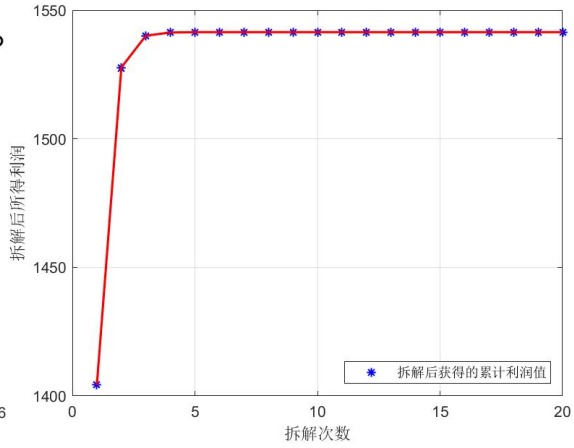
[3]姜启源等编，数学建模案例选集[M](第 1 版)，北京，高等教育出版社，2006.1.

附录

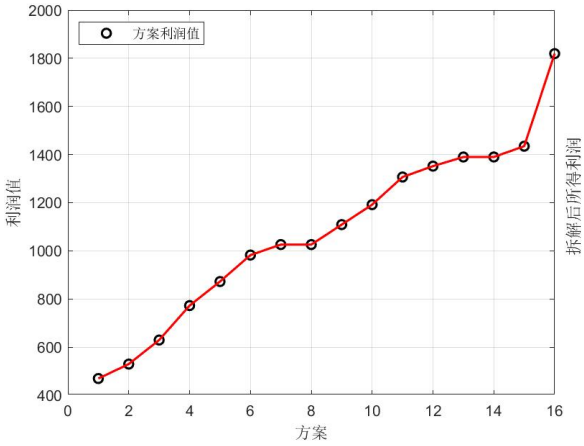
附录 1：问题二中情况 2-情况 6 的 利润变化图 和 拆解次数与利润的关系图

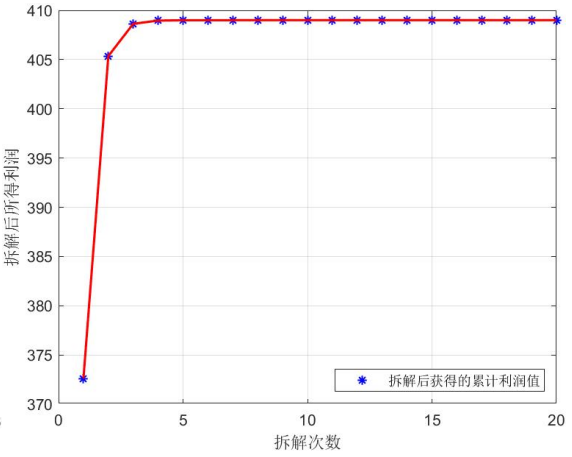
情况 2：



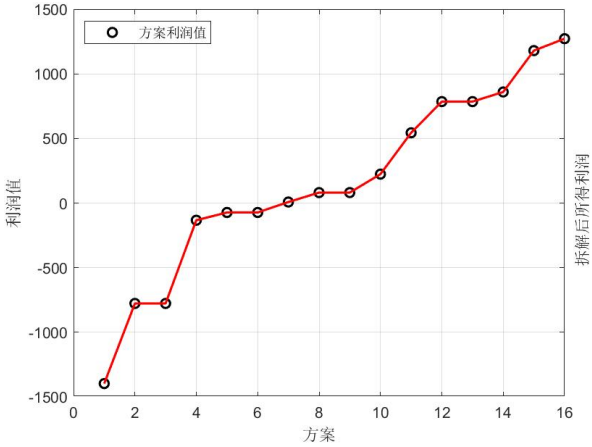


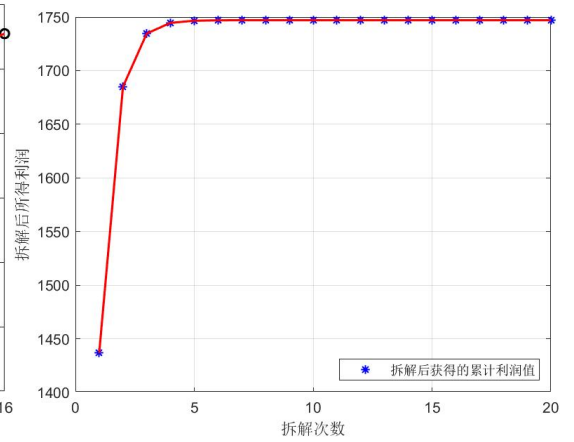
情况 3:



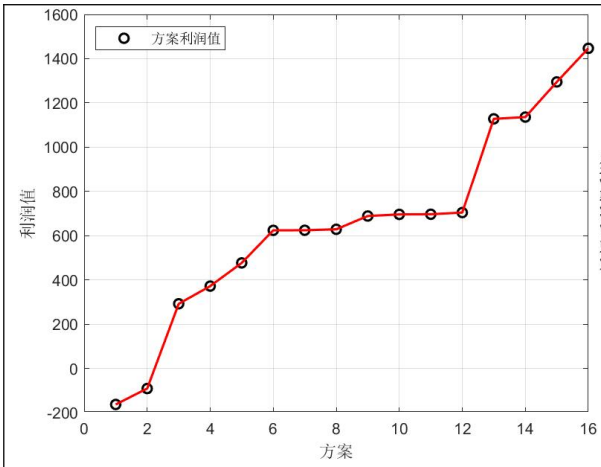


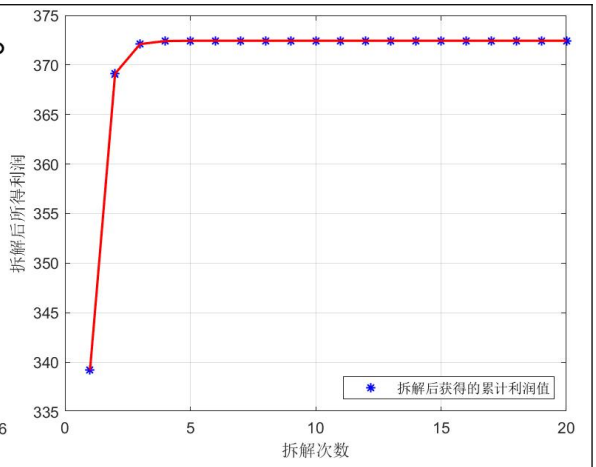
情况 4:



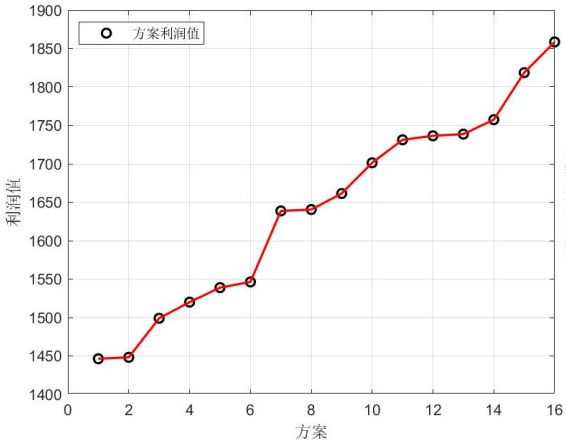


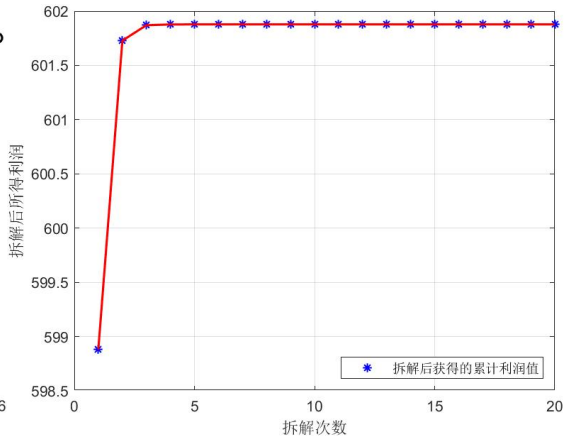
情况 5:





情况 6:





附录 2：该代码 Matlab 编写的，作用是求解问题二中的情况 1（剩余 5 中情况仅需根据表 1 进行部分参数的修改即可）
%% 清理 clc, clear; % cost1 零配件1的购买和检测成本 % cost2 零配件2的购买和检测成本 % cost3 第一次生成成品的装配和检测成本 % cost4 第一次拆解不合格成品的拆解成本 % cost5 拆解后零配件1的检测成本 % cost6 拆解后零配件2的检测成本 % change_fee 调换成本 % f1 拆解后的利润 %% 参数设置 自行设置情况 Buy1=4; Buy2=18; % 购买单价固定 sale=56; % 市场售价固定

```
produce=6; % 装配成本固定
% 次品率、检测成本
D_rate1=0.1;
detection1=2;
D_rate2=0.1;
detection2=3;
D_ratep=0.1;
detection_p=3;
% 销售额、成本初始化
sale_all=0; cost_all=0; change_fee=0;
% 调换费用、拆解费用
dismantle=5;
change=6;
n=input('请输入购买数量n：');
%% 解的设置
X=Q2();
%% 外层循环
for j=1:16
    x=X(j,:);
    % 参数重置
    change_fee=0; cost1=0; cost2=0; cost3=0; cost4=0; cost5=0; cost6=0; f1=0;
    disp(['方案',int2str(j)]);
    %% 零配件1
    if x(1)==1 % 检测
        cost1=(Buy1+detection1)*n;
        great_goods1=n*(1-D_rate1);
        goods1=great_goods1;
        bad_goods1=0;
        great_rate1=1; bad_rate1=0;
    else % 不检测
        cost1=Buy1*n;
        great_goods1=n*(1-D_rate1);
        goods1=n;
        bad_goods1=n*D_rate1;
        great_rate1=1-D_rate1; bad_rate1=D_rate1;
    end
    %% 零配件2
    if x(2)==1 % 检测
```

```
        cost2=(Buy2+detection2)*n;
        great_goods2=n*(1-D_rate2);
        goods2=great_goods2;
        bad_goods2=0;
        great_rate2=1; bad_rate2=0;
    else % 不检测
        cost2=Buy2*n;
        great_goods2=n*(1-D_rate2);
        goods2=n;
        bad_goods2=n*D_rate2;
        great_rate2=1-D_rate2; bad_rate2=D_rate2;
    end
    %% 比对goods数量
    if goods1>=goods2
        production=goods2;
    else
        production=goods1;
    end
    %% 得出真正成品次品率
D_ratep_real=great_rate1*great_rate2*D_ratep+great_rate1*bad_rate2+bad_rate1*great_rate2+bad_rate
1*bad_rate2;
    %% 成品
    if x(3)==1 % 检测
        cost3=(produce+detection_p)*production;
        bad_production=production*D_ratep_real;
        great_production=production-bad_production;
    else
        bad_production=production*D_ratep_real;
        great_production=production-bad_production;
        change_fee=bad_production*change;
        cost3=produce*production;
    end
    %% 拆解
    if x(4)==1 % 拆解
        cost4=bad_production*dismantle;
        if x(1)==0 % 零配件1没有检测
            cost5=bad_production*detection1;
            new_goods1=bad_production*(1-D_rate1);
```

```
else
    cost5=0;
    new_goods1=bad_production;
end
if x(2)==0 % 零配件2没有检测
    cost6=bad_production*detection2;
    new_goods2=bad_production*(1-D_rate2);
else
    cost6=0;
    new_goods2=bad_production;
end
if new_goods1>=new_goods2
    new_production=new_goods2;
else
    new_production=new_goods1;
end
% 拆解利润函数是收敛的
syms k;

  f(k)=sale*new_production*(1-D_ratep)*(1-D_ratep^k)/(1-D_ratep)-(produce+detection_p)*
  new_production*(1-D_ratep^k)/(1-D_ratep)-dismantle*new_production*D_ratep*(1-D_rate
  p^(k-1))/(1-D_ratep);
for i=1:20
    Y(i)=f(i);
end
plot(1:20,Y,'b*','linewidth',1);
hold on
plot(1:20,Y,'r-','linewidth',1.5);
xlabel('拆解次数'); ylabel('拆解后所得利润');
legend('拆解后获得的累计利润值','location','southeast');
grid on
hold off
% 输入拆解利润最大值时迭代次数k
k=input('根据图象输入利润函数最大时的迭代次数k：');
% 拆解利润函数

  f1=sale*new_production*(1-D_ratep)*(1-D_ratep^k)/(1-D_ratep)-(produce+detection_p)*n
  ew_production*(1-D_ratep^k)/(1-D_ratep)-dismantle*new_production*D_ratep*(1-D_ratep
```

```
            ^(k-1))/(1-D_ratep);
        % 利润
        cost_all=cost1+cost2+cost3+cost4+cost5+cost6+change_fee;
        sale_all=great_production*sale+f1;
        M=sale_all-cost_all;
    else % 不拆解
        cost_all=cost1+cost2+cost3+change_fee;
        sale_all=great_production*sale;
        M=sale_all-cost_all;
    end
%% 收集数据
m(j)=M;
end
% 寻找最值并排序
index_x=find(max(m)==m);
m=sort(m);
index=find(max(m)==m); % 最大利润所在索引
%% 画图
figure(2)
plot(1:16,m,'ko','LineWidth', 1.5);
hold on
plot(1:16,m,'r-','LineWidth', 1.5);
xlabel('方案'); ylabel('利润值');
legend('方案利润值','Location','northwest');
grid on
%% 输出
disp('-----------------------------------------------');
disp('重新排序中...');
disp(['最大利润为：',int2str(m(index))]);
disp(['此时的方案为：',int2str(X(index_x,:))]);

function x=Q2()
% 共有2*2*2*2=16种组合方式，故可以采用枚举法。
% 若组合方式过多，可使用模拟退火算法搜索最优决策方案。
x=[0,0,0,0;0,0,0,1;0,0,1,0;0,0,1,1;0,1,0,0;0,1,0,1;0,1,1,0;0,1,1,1;1,0,0,0;1,0,0,1;1,0,1,0;1,0,1,1;1,1,0,0
    1,1,0,1;1,1,1,0;1,1,1,1];
end
```

附录 3：该代码是 Matlab 编写的，作用是绘制不同情况下不同方案的销售额和总成本变化图

```
clc;
clear;
%% 本代码用于处理图像
[M1,path1,Sale1,Cost1]=Qimage(0.1,2,0.1,3,0.1,3,6,5,100);
[M2,path2,Sale2,Cost2]=Qimage(0.2,2,0.2,3,0.1,3,6,5,100);
[M3,path3,Sale3,Cost3]=Qimage(0.1,2,0.1,3,0.1,3,30,5,100);
[M4,path4,Sale4,Cost4]=Qimage(0.2,1,0.2,1,0.2,2,30,5,100);
[M5,path5,Sale5,Cost5]=Qimage(0.1,8,0.2,1,0.1,2,10,5,100);
[M6,path6,Sale6,Cost6]=Qimage(0.05,2,0.05,3,0.05,3,10,40,100);
% M为最大利润 path为最优 Sale为每个方案的销售额 Cost为每个方案的成本
%% 数据整理
figure(1) % 不同情况的利润值比较
M=[M1,M2,M3,M4,M5,M6];
X=1:6;
plot(X,M,'r*','LineWidth',1);
hold on
plot(X,M,'k--');
grid on
xlabel('情况');ylabel('利润值');
legend('不同情况的最大利润值');
figure(2) % 六种情况的销售额随方案变化
X2=1:16;
set(gcf,'Position',[100 100 1000 600]);
plot(X2,Sale1,'ro-',LineWidth=3);
hold on
plot(X2,Sale2,'bo-',LineWidth=1.5);
plot(X2,Sale3,'co-',LineWidth=1.5);
plot(X2,Sale4,'go-',LineWidth=1.5);
plot(X2,Sale5,'ko-',LineWidth=1.5);
plot(X2,Sale6,'mo-',LineWidth=1.5);
grid on
xlabel('方案');ylabel('销售额');
legend('方案1的销售值','方案2的销售值','方案3的销售值','方案4的销售值','方案5的销售值','方案6的销售值' ...
    ,Location='southeast');
figure(3)
```

```
set(gcf,'Position',[100 100 1000 600]);
plot(X2,Cost1,'ro-',LineWidth=1.5);
hold on
plot(X2,Cost2,'bo-',LineWidth=1.5);
plot(X2,Cost3,'go-',LineWidth=1.5);
plot(X2,Cost4,'co-',LineWidth=1.5);
plot(X2,Cost5,'ko-',LineWidth=1.5);
plot(X2,Cost6,'mo-',LineWidth=1.5);
grid on
xlabel('方案');ylabel('总成本');
legend('方案1的总成本','方案2的总成本','方案3的总成本','方案4的总成本','方案5的总成本','方案6的总成本' ...
    ,Location='northeast');
function [a,b,Sale,Cost]=Qimage(D_rate1,detection1,D_rate2,detection2,D_ratep,detection_p,change,dismantle,n)
% cost1  零配件1的购买和检测成本
% cost2  零配件2的购买和检测成本
% cost3  第一次生成成品的装配和检测成本
% cost4  第一次拆解不合格成品的拆解成本
% cost5  拆解后零配件1的检测成本
% cost6  拆解后零配件2的检测成本
% change_fee  调换成本
% f1  拆解后的利润
%% 参数设置  自行设置情况
Buy1=4; Buy2=18; %  购买单价固定
sale=56; %  市场售价固定
produce=6; %  装配成本固定
%  销售额、成本初始化
sale_all=0; cost_all=0; change_fee=0;
%% 解的设置
X=Q2();
%% 外层循环
for j=1:16
    x=X(j,:);
    %  参数重置
    change_fee=0; cost1=0; cost2=0; cost3=0; cost4=0; cost5=0; cost6=0; f1=0;
    disp(['方案',int2str(j)]);
```

```
    %% 零配件1
    if x(1)==1 % 检测
        cost1=(Buy1+detection1)*n;
        great_goods1=n*(1-D_rate1);
        goods1=great_goods1;
        bad_goods1=0;
        great_rate1=1; bad_rate1=0;
    else % 不检测
        cost1=Buy1*n;
        great_goods1=n*(1-D_rate1);
        goods1=n;
        bad_goods1=n*D_rate1;
        great_rate1=1-D_rate1; bad_rate1=D_rate1;
    end
    %% 零配件2
    if x(2)==1 % 检测
        cost2=(Buy2+detection2)*n;
        great_goods2=n*(1-D_rate2);
        goods2=great_goods2;
        bad_goods2=0;
        great_rate2=1; bad_rate2=0;
    else % 不检测
        cost2=Buy2*n;
        great_goods2=n*(1-D_rate2);
        goods2=n;
        bad_goods2=n*D_rate2;
        great_rate2=1-D_rate2; bad_rate2=D_rate2;
    end
    %% 比对goods数量
    if goods1>=goods2
        production=goods2;
    else
        production=goods1;
    end
    %% 得出真正成品次品率
D_ratep_real=great_rate1*great_rate2*D_ratep+great_rate1*bad_rate2+bad_rate1*great_rate2+bad_rate1*bad_rate2;
    %% 成品
```

```
    if x(3)==1 % 检测
        cost3=(produce+detection_p)*production;
        bad_production=production*D_ratep_real;
        great_production=production-bad_production;
    else
        bad_production=production*D_ratep_real;
        great_production=production-bad_production;
        change_fee=bad_production*change;
        cost3=produce*production;
    end
    %% 拆解
    if x(4)==1 % 拆解
        cost4=bad_production*dismantle;
        if x(1)==0 % 零配件1没有检测
            cost5=bad_production*detection1;
            new_goods1=bad_production*(1-D_rate1);
        else
            cost5=0;
            new_goods1=bad_production;
        end
        if x(2)==0 % 零配件2没有检测
            cost6=bad_production*detection2;
            new_goods2=bad_production*(1-D_rate2);
        else
            cost6=0;
            new_goods2=bad_production;
        end
        if new_goods1>=new_goods2
            new_production=new_goods2;
        else
            new_production=new_goods1;
        end
        % 拆解利润函数是收敛的
        syms k;

f(k)=sale*new_production*(1-D_ratep)*(1-D_ratep^k)/(1-D_ratep)-(produce+detection_p)*new_produc
tion*(1-D_ratep^k)/(1-D_ratep)-dismantle*new_production*D_ratep*(1-D_ratep^(k-1))/(1-D_ratep);
        for i=1:20
```

```
            Y(i)=f(i);
        end
        % 输入拆解利润最大值时迭代次数k
        % k=input('根据图象输入利润函数最大时的迭代次数k：');
        % 拆解利润函数
        k=4;

f1=sale*new_production*(1-D_ratep)*(1-D_ratep^k)/(1-D_ratep)-(produce+detection_p)*new_producti
on*(1-D_ratep^k)/(1-D_ratep)-dismantle*new_production*D_ratep*(1-D_ratep^(k-1))/(1-D_ratep);
        % 利润
        cost_all=cost1+cost2+cost3+cost4+cost5+cost6+change_fee;
        sale_all=great_production*sale+f1;
        M=sale_all-cost_all;
    else % 不拆解
        cost_all=cost1+cost2+cost3+change_fee;
        sale_all=great_production*sale;
        M=sale_all-cost_all;
    end
%% 收集数据
m(j)=M;
Sale(j)=sale_all;
Cost(j)=cost_all;
end
% 寻找最值并排序
index_x=find(max(m)==m);
m=sort(m);
index=find(max(m)==m); % 最大利润所在索引
a=m(index);
b=X(index_x,:);
disp('----------------完成------------------');
end
function x=Q2()
% 共有2*2*2*2=16种组合方式，故可以采用枚举法。
% 若组合方式过多，可使用模拟退火算法搜索最优决策方案。
x=[0,0,0,0;0,0,0,1;0,0,1,0;0,0,1,1;0,1,0,0;0,1,0,1;0,1,1,0;0,1,1,1;1,0,0,0;1,0,0,1;1,0,1,0;1,0,1,1;1,1,0,0
    1,1,0,1;1,1,1,0;1,1,1,1];
end
```

附录 4：该代码是 Matlab 编写的，作用是求解模型三

```
%% 清除一切障碍
clc, clear;
% path数组中1-8为配件是否检查，9、10为半成品1是否检查、拆解，11、12为半成品2是否检查、拆解，13、14为半成品3是否检查、拆解，15、16为成品是否检查、拆解
rng(1);
%% 模拟退火参数初始化
T0 = 1000;     % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为T0
maxgen = 5000;   % 设置最大迭代次数
Lk = 1000;   % 变化后不同温度下的迭代次数
beta = 0.95;   % 设置温度衰减系数
%% 条件初始化
n=input('输入购买配件数量n：');
B=[2,8,12,2,8,12,8,12];
D=0.1;
d=[1,1,2,1,1,2,1,2];
M_all=[];
%%   随机生成初始解
path = randi([0,1],1,16);   % 生产初始的路径path0，随后不断改进
best_path=path; % 默认初始路径为最优路径
result_end=0; result=0;
%% 模拟退火过程
for iter = 1 : maxgen   % 外循环iter, 采用指定最大迭代次数
    for i = 1 : Lk   %   内循环，即在每个温度下开始迭代，整体共迭代maxgen*Lk次
        % 参数初始化
        cost_all=0; sale_all=0;
        path1=new_path(path);
        for j=1:8
%% 基础配件的购买和检查
            if path1(j)==1 % 配件检测
                cost_all=cost_all+(B(j)+d(j))*n;
                goods(j)=(1-D)*n;
                great_rate(j)=1;
                bad_rate(j)=0;
            else
                cost_all=cost_all+B(j)*n;
```

```
                great_goods(j)=(1-D)*n;
                bad_goods(j)=D*n;
                goods(j)=n;
                great_rate(j)=1-D;
                bad_rate(j)=D;
            end
        end
        use_goods=min(goods);
        if length(use_goods)==8
            use_goods=use_goods(1);
        end
        % 半成品实际次品率
        half_goods_D_real(1)=1-great_rate(1)*great_rate(2)*great_rate(3)*(1-D);
        half_goods_D_real(2)=1-great_rate(4)*great_rate(5)*great_rate(6)*(1-D);
        half_goods_D_real(3)=1-great_rate(7)*great_rate(8)*(1-D);
        % 半成品组成成分
        bad_half(1)=use_goods*half_goods_D_real(1);
        great_half(1)=use_goods-bad_half(1);
        bad_half(2)=use_goods*half_goods_D_real(2);
        great_half(2)=use_goods-bad_half(2);
        bad_half(3)=use_goods*half_goods_D_real(3);
        great_half(3)=use_goods-bad_half(3);
%% 半成品的检测与拆解
        if path1(9)==1 % 半成品1检测
            cost_all=cost_all+12*use_goods;
            use_half1=great_half(1);
            great_half_rate(1)=1;
            bad_half_rate(1)=0;
            if path1(10)==1 % 半成品1拆解
                cost_all=cost_all+6*bad_half(1);
                if path1(1)==1 % 零部件1检测过
                    production(1)=bad_half(1);
                else
                    cost_all=cost_all+bad_half(1)*(1-D)*d(1);
                    production(1)=bad_half(1)*(1-D);
                end
                if path1(2)==1 % 零部件2检测过
                    production(2)=bad_half(1);
```

```
            else
                cost_all=cost_all+bad_half(1)*(1-D)*d(2);
                production(2)=bad_half(1)*(1-D);
            end
            if path1(3)==1 % 零部件3检测过
                production(3)=bad_half(1);
            else
                cost_all=cost_all+bad_half(1)*(1-D)*d(3);
                production(3)=bad_half(1)*(1-D);
            end
            production_half(1)=min([production(1),production(2),production(3)]);
            cost_all=cost_all+6*production_half(1)*((1-0.1^5)/9+2*(1-0.1^7)/0.9);
            use_half1=use_half1+production_half(1);
        end
    else % 不检测半成品1
        cost_all=cost_all+8*use_goods;
        use_half1=use_goods;
        great_half_rate(1)=1-D;
        bad_half_rate(1)=D;
    end
    if path1(11)==1 % 半成品2检测
        cost_all=cost_all+12*use_goods;
        use_half2=great_half(2);
        great_half_rate(2)=1;
        bad_half_rate(2)=0;
        if path1(12)==1 % 半成品2拆解
            cost_all=cost_all+6*bad_half(2);
            if path1(4)==1 % 零部件4检测过
                production(4)=bad_half(2);
            else
                cost_all=cost_all+bad_half(2)*(1-D)*d(4);
                production(4)=bad_half(2)*(1-D);
            end
            if path1(5)==1 % 零部件5检测过
                production(5)=bad_half(2);
            else
                cost_all=cost_all+bad_half(2)*(1-D)*d(5);
                production(5)=bad_half(2)*(1-D);
```

```
            end
            if path1(6)==1 % 零部件6检测过
                production(6)=bad_half(2);
            else
                cost_all=cost_all+bad_half(2)*(1-D)*d(6);
                production(6)=bad_half(2)*(1-D);
            end
            production_half(2)=min([production(4),production(5),production(6)]);
            cost_all=cost_all+6*production_half(2)*((1-0.1^5)/9+2*(1-0.1^7)/0.9);
            use_half2=use_half2+production_half(2);
        end
    else % 不检测半成品2
        cost_all=cost_all+8*use_goods;
        use_half2=use_goods;
        great_half_rate(2)=1-D;
        bad_half_rate(2)=D;
    end
    if path1(13)==1 % 半成品3检测
        cost_all=cost_all+12*use_goods;
        use_half3=great_half(3);
        great_half_rate(3)=1;
        bad_half_rate(3)=0;
        if path1(14)==1 % 半成品3拆解
            cost_all=cost_all+6*bad_half(3);
            if path1(7)==1 % 零部件7检测过
                production(7)=bad_half(3);
            else
                cost_all=cost_all+bad_half(3)*(1-D)*d(7);
                production(7)=bad_half(3)*(1-D);
            end
            if path1(8)==1 % 零部件8检测过
                production(8)=bad_half(3);
            else
                cost_all=cost_all+bad_half(3)*(1-D)*d(8);
                production(8)=bad_half(3)*(1-D);
            end
            production_half(3)=min([production(7),production(8)]);
            cost_all=cost_all+6*production_half(3)*((1-0.1^5)/9+2*(1-0.1^7)/0.9);
```

```
                use_half3=use_half3+production_half(3);
            end
        else % 不检测半成品3
            cost_all=cost_all+8*use_goods;
            use_half3=use_goods;
            great_half_rate(3)=1-D;
            bad_half_rate(3)=D;
        end
        use_half=min([use_half1,use_half2,use_half3]); % use_half 能够做成成品的最少半成品数量
%% 成品实际次品率计算
        real_cp_Drate=1-great_half_rate(1)*great_half_rate(2)*great_half_rate(3)*(1-D);
        real_great_cp=use_half*(1-real_cp_Drate); % 正品成品数量
        real_bad_cp=use_half-real_great_cp; % 次品成品数量
%% 成品制造
        if path1(15)==1 % 检查成品
            cost_all=cost_all+14*use_half;
        else % 不检查成品
            cost_all=cost_all+8*use_half+40*real_bad_cp;
        end
        % 拆解选择
        if path1(16)==0 % 不拆解不合格成品
            sale_all=200*real_great_cp;
            M=sale_all-cost_all;
        else % 拆解不合格成品
            cost_all=cost_all+10*real_bad_cp;
            sale_all=200*real_great_cp;
            if path1(9)==0 % 半成品1没有检测过
                cost_all=cost_all+real_bad_cp*4;
                great_bcp1=real_bad_cp*(1-half_goods_D_real(1));
            else
                great_bcp1=real_bad_cp;
            end
            if path1(11)==0 % 半成品2没有检测过
                cost_all=cost_all+real_bad_cp*4;
                great_bcp2=real_bad_cp*(1-half_goods_D_real(2));
            else
                great_bcp2=real_bad_cp;
```

```
                end

                if path1(13)==0 %  半成品3没有检测过
                    cost_all=cost_all+real_bad_cp*4;
                    great_bcp3=real_bad_cp*(1-half_goods_D_real(3));
                else
                    great_bcp3=real_bad_cp;
                end
                %  选择最少可用半成品
                great_bcp=min([great_bcp1,great_bcp2,great_bcp3]);
                % f1为选择拆卸后可获得的利润

f1=200*great_bcp*(1-D)*(1-D^6)/(1-D)-(12)*great_bcp*(1-D^6)/(1-D)-10*great_bcp*D*(1-D^5)/(1-D)
;
                sale_all=sale_all+f1;
                M=sale_all-cost_all;
            end
            z(i)=M;
            %  一定程度上接受新解
            if M > result
                path = path1; %  更新当前路径为新路径
                result = M; %  更新
            else
                p = exp(-(M - result)/T); %  根据Metropolis准则计算一个可接受概率
                if rand(1) < p     %  生成一个随机数和这个概率比较，如果该随机数小于这个概率则
接受新解
                    path = path1;   %  更新当前路径为新路径
                    result = M; %  更新
                end
            end
            %  修改路径   找最大值和路径
            if result_end < result
                result_end = result;
                best_path = path;
            end
        end
        %  记录每次内循环后的最大值，画图观察是否收敛
        RESULT(iter) = result_end;
```

```
    T = beta*T;     % 温度下降法则
end
disp(['本次最大利润为:',int2str(result_end)]);
disp(['本次最优方案为:',int2str(best_path)]);
%% 画图
figure(1)
plot(1:maxgen,RESULT,'r-','LineWidth',1.5); % 自变量为迭代次数1:maxgen，因变量RESULT
xlabel('迭代次数'); % x轴名称
ylabel('最大利润值'); % y轴名称
legend('降温后利润最大值收敛曲线','Location','southeast');
grid on;
z=sort(z);
figure(2)
plot(1:Lk,z,'b-','LineWidth',1.5);
xlabel('迭代次数'); % x轴名称
ylabel('利润值'); % y轴名称
legend('某温度下不同方案的利润值变化曲线','Location','northwest');
grid on;
figure(3)
syms k;
f(k)=200*great_bcp*(1-D)*(1-D^k)/(1-D)-(12)*great_bcp*(1-D^k)/(1-D)-10*great_bcp*D*(1-D^(k-1))/
(1-D);
for i=1:20
    Y(i)=f(i);
end
plot(1:20,Y,'b*','linewidth',1);
hold on
plot(1:20,Y,'r-','linewidth',1.5);
xlabel('拆解次数'); ylabel('拆解后所得利润');
legend('拆解后获得的累计利润值','location','southeast');
grid on
hold off

function path1 = new_path(path0) % path0: 代入原来的路径
    n = length(path0); % n为点数量
    % 随机选择两种生成新路径的方法
    p1 = 0.33;   % 自行设置使用交换法产生新路径的概率
    p2 = 0.33;   % 自行设置使用移位法产生新路径的概率
```

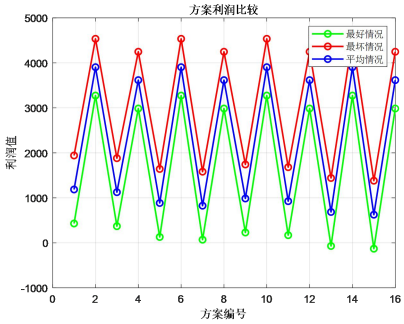
```
    r = rand(1); % 生成一个[0 1]间均匀分布的随机数
    % 方法一
    if   r< p1       % 用交换法产生新路径
        c1 = randi(n);     % 产生1-n中的一个随机整数
        c2 = randi(n);     % 产生1-n中的一个随机整数
        path1 = path0;   % 将path0的值给path1
        path1(c1) = path0(c2);   %改path1第c1个位置的元素为path0第c2个位置的元素
        path1(c2) = path0(c1);   %改path1第c2个位置的元素为path0第c1个位置的元素
    % 方法二
    elseif r < p1+p2 % 使用移位法产生新路径
        c1 = randi(n);     % 产生1-n中的一个随机整数
        c2 = randi(n);     % 产生1-n中的一个随机整数
        c3 = randi(n);     % 产生1-n中的一个随机整数
        sort_c = sort([c1 c2 c3]);   % 对c1 c2 c3从小到大进行排序
        c1 = sort_c(1);   c2 = sort_c(2);   c3 = sort_c(3);   % c1 < = c2 <=   c3
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:c3);
        tem4 = path0(c3+1:end);
        path1 = [tem1 tem3 tem2 tem4];
    % 方法三
    else   % 使用倒置法产生新路径
        c1 = randi(n);     % 产生1-n中的一个随机整数
        c2 = randi(n);     % 产生1-n中的一个随机整数
        if c1>c2   % 若c1比c2大，就交换c1和c2的值
            tem = c2;
            c2 = c1;
            c1 = tem;
        end
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:end);
        path1 = [tem1 fliplr(tem2) tem3];     %矩阵的左右对称翻转 fliplr，上下对称翻转 flipud
    end
end
```

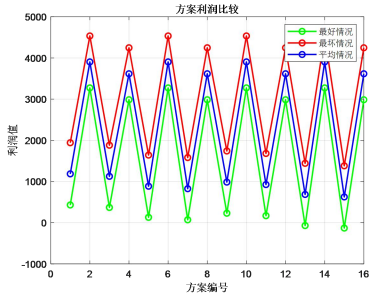
附录 5：该代码是 Matlab 编写的，作用是计算次品率的置信区间及可视化模拟

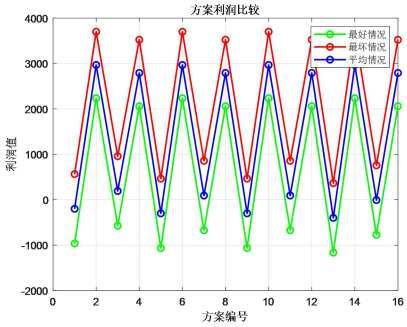
```
%% 抽样模拟
% 参数设置
p_estimated = 0.1;            % 估计的次品率
error_margin = 0.02;          % 允许的误差范围
confidence_level = 0.99;   % 置信水平（99%）
% 调用函数并绘制置信区间分布图
shouldDetect = sample_estimation(p_estimated, error_margin, confidence_level);
%% 计算次品率的置信区间及可视化模拟
function shouldDetect = sample_estimation(p_estimated, error_margin, confidence_level)
    % 参数定义
    Z = norminv(1 - (1 - confidence_level) / 2);   % Z值，对应99%置信水平
    n_required = (Z^2 * p_estimated * (1 - p_estimated)) / (error_margin^2);   % 计算所需样本量
    % 打印所需的最小样本量
    fprintf('所需的最小样本量: %.2f\n', n_required);
    % 假设样本结果: 用估计次品率生成假设样本的次品率范围
    n = ceil(n_required);   % 样本数量（取整）
    sample_defects = binornd(n, p_estimated, [100000, 1]);   % 生成大量随机样本次品率数据（模拟抽样100000次）
    p_sample = sample_defects / n;   % 样本次品率
    % 计算样本次品率的置信区间
    lower_bound = p_estimated - Z * sqrt(p_estimated * (1 - p_estimated) / n); % 区间下限
    upper_bound = p_estimated + Z * sqrt(p_estimated * (1 - p_estimated) / n); % 区间上限
    fprintf('估计次品率置信区间: [%.4f, %.4f]\n', lower_bound, upper_bound);
    % 决策规则: 如果置信区间上限超过可接受次品率，则需要进行检测
    acceptable_defect_rate = 0.10;   % 设定企业的可接受次品率为10%（标称值）
    % 决策判断
    if upper_bound > acceptable_defect_rate
        fprintf('检测决策: 建议进行检测。\n');
        shouldDetect = true;
    else
        fprintf('检测决策: 不需要检测。\n');
        shouldDetect = false;
    end
    %% 可视化
    figure;
    % 绘制样本次品率的直方图
    histogram(p_sample, 50, 'Normalization', 'probability', 'FaceColor', [0.2 0.4 0.8]);
    hold on;
```

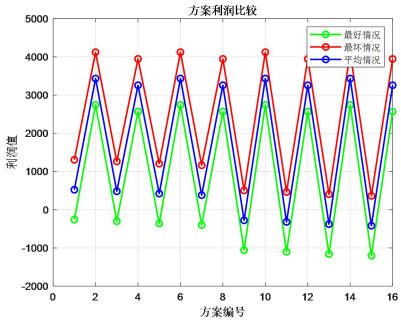
```
    % 红线为置信区间线
    y_limits = ylim;
    line([lower_bound, lower_bound], y_limits, 'Color', 'r', 'LineWidth', 2);   % 置信区间下界
    line([upper_bound, upper_bound], y_limits, 'Color', 'r', 'LineWidth', 2);   % 置信区间上界
    % 添加图例和标签
    title('次品率分布及其置信区间', 'FontSize', 14);
    xlabel('次品率', 'FontSize', 12);
    ylabel('频率', 'FontSize', 12);
    % 黄色虚线为可接受次品率的参考线
    xline(acceptable_defect_rate, '--y', 'LineWidth', 2, 'LabelHorizontalAlignment', 'left');
    hold off;
end
```

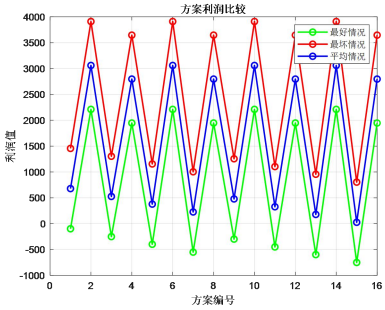
附录 6：问题四（1）不同情况不同方案利润的比较图











附录 7：支撑材料文件列表
1.Q1 文件中包含问题**1**的代码和图形 **2.Q2** 文件中包含问题**2**的两种方法的代码、图形和结果 **3.Q3** 文件中包含问题**3**的代码、图形和结果 **4.Q4** 文件中包含问题**4**的代码和结果，由于图形较多不再单独保存